

get()ing the pointer from optional

Document #: P4189R0 [[Latest](#)] [[Status](#)]
Date: 2026-05-12
Project: Programming Language C++
Audience: LEWG
Reply-to: Nevin “:-)” Liber
<nliber@anl.gov>

Contents

- [1 Revision History](#)
 - [1.1 R0](#)
- [2 Introduction](#)
- [3 Motivation and Scope](#)
- [4 Impact On the Standard](#)
- [5 Design Decisions](#)
 - [5.1 Why `.get\_ptr\(\)` ?](#)
 - [5.2 Why not the other choices?](#)
- [6 Technical Specifications](#)
 - [6.1 Add to `\[optional.optional.general\]`:](#)
 - [6.2 Add to `\[optional.observe\]`:](#)
 - [6.3 Add to `\[optional.optional.ref.general\]`:](#)
 - [6.4 Add to `\[optional.ref.observe\]`:](#)
 - [6.5 Modify in `\[version.syn\]`:](#)
- [7 Acknowledgements](#)
- [8 References](#)

§ 1 Revision History

§ 1.1 R0

First proposed.

§ 2 Introduction

There is no easy, obvious way to retrieve the pointer stored in an `optional<T&>`, nor a pointer to the object in an `optional<T>`. This proposal rectifies that by proposing a `.get_ptr()` method.

§ 3 Motivation and Scope

When calling a C or a legacy C++ API, a raw pointer is often used instead of an `optional<T&>` (in the case of C or pre-C++26 APIs) or an `optional<T>` (in the case of C or pre-C++17 APIs). This paper proposes making it easy for developers to convert an `optional` into a pointer by adding a `.get_ptr()` member function.

During the discussion around changing the return type for `inplace_vector<T, N>::try*_back()` from a `T*` to a `std::optional<T&>`, it was noted that there is no easy way to convert the `optional<T&>` returned to a `T*`.

For instance, if one wants a one-liner to convert the result of `iv.try_emplace_back(/* ... */) to a pointer without storing the optional<T&> in a variable, one would have to write something like the non-obvious (courtesy of Claude Sonnet 4.6):`

```
iv.try_emplace_back(/* ... */).transform([](auto& r) { return std::address-  
sof(r); }).value_or(nullptr);
```

While there are various tradeoffs that can be made to simplify this under some circumstances (use a two-liner by storing the result first, assume that the address-of operator is not overloaded, specify the type `T` being used, etc.), that is not the point.

There should be an easy way to convert an `optional` into a pointer, where a pointer with a null pointer value corresponds directly to a disengaged `optional`. Dereferencing a non-null pointer, like dereferencing an engaged `optional`, produces a reference to the object. This paper proposes following the precedent set by `Boost.Optional`.

§ 4 Impact On the Standard

Other than bumping up the value of a feature test macro, this is purely additive.

§ 5 Design Decisions

The C++ Standard Library, as well as third party libraries, have other types that wrap pointers. Most of those types provide an easy way to retrieve the pointer. Here is an (incomplete) list of such types and their pointer accessors:

Type	Pointer
<code>optional<T&></code>	<code>.transform([](auto& r) { return std::addressof(r); }).value_or(nullptr);</code>
<code>::boost::optional<T&></code>	<code>.get_ptr()</code> // <i>T* to object referred to in the optional</i>
<code>::boost::optional<T></code>	<code>.get_ptr()</code> // <i>T* to object stored in the optional</i>
<code>reference_wrapper<T></code>	<code>std::addressof(rw.get())</code> // <i>T*</i>
<code>unique_ptr<T, D></code> <code>shared_ptr<T></code> <code>experimental::observer_ptr<T></code> <code>::gsl::not_null<T*></code> <code>::boost::intrusive_ptr<T></code> <code>::boost::shared_array<T></code> <code>::boost::scoped_array<T></code>	<code>.get()</code> // <i>T*</i>
<code>basic_string_view<T, CT></code> <code>span<T, E></code>	<code>.data()</code> // <i>T*</i>
<code>contiguous_iterator</code>	<code>std::to_address(ci)</code> // <i>T*</i>
<code>weak_ptr<T></code>	<code>.lock().get()</code> // <i>T*</i>
<code>atomic_ref<T></code>	<code>.address()</code> // <i>cv void*</i>

Of those, the best fit is making the same choice as Boost.Optional.

§ 5.1 Why `.get_ptr()` ?

The primary use case, as mentioned, is to use this with C APIs and legacy C++ APIs that use pointers because there is/was (C/C++) no suitable vocabulary type to use. `.get_ptr()` can be used for both `optional<T>` & `optional<T&>`. Other names have other baggage, while this name has the added benefit of being consistent with Boost.Optional.

§ 5.2 Why not the other choices?

`.get()`

It is obvious from the name of the type (`*_ptr` , `*_array` , etc.) that the type holds a pointer. That is not true about `optional<T&>` .

The other use of `.get()` is in `reference_wrapper` , where it returns a reference, not a pointer.

```
.data()
```

For an empty `span` or `basic_string_view` (the moral equivalent of a detached `optional`), there is no guarantee that the pointer returned has a null pointer value, so this would be the first time we are making such a guarantee. Heck, even `array<T, 0>::data()` doesn't guarantee that, and that is always empty.

Additionally, `.data()` is typically used in conjunction with `.size()` , which is not a part of `optional` . Even if `.size()` is added in the future, its use would still needlessly complicate extracting the pointer.

```
std::to_address(/* ... */)
```

Because a *contiguous-iterator* could already be a raw pointer, it makes sense for that to use a free function. A member function makes more sense for `optional` as it doesn't have that same complication.

```
.lock().get()
```

You have to lock the `weak_ptr` to guarantee the pointer returned is valid. We do not have that complication with `optional` .

```
.address() // cv void*
```

`atomic_ref` returns a `cv void*` because most naïve uses of the returned pointer are bugs. This is not true of `optional` .

§ 6 Technical Specifications

These are relative to N5032:

§ 6.1 Add to `[optional.optional.general]`:

```
// observers
constexpr const T* operator->() const noexcept;
constexpr T* operator->() noexcept;
constexpr const T& operator*() const & noexcept;
constexpr T& operator*() & noexcept;
constexpr T&& operator*() && noexcept;
constexpr const T&& operator*() const && noexcept;
constexpr explicit operator bool() const noexcept;
constexpr bool has_value() const noexcept;
constexpr const T& value() const &; // freestanding-deleted
constexpr T& value() &; // freestanding-deleted
```

```
constexpr T&& value() &&; // freestanding-deleted
constexpr const T&& value() const &&; // freestanding-deleted
template<class U = remove_cv_t<T>> constexpr T value_or(U&&) const &;
template<class U = remove_cv_t<T>> constexpr T value_or(U&&) &&;
constexpr T* get_ptr() noexcept;
constexpr const T* get_ptr() const noexcept;
```

§ 6.2 Add to [optional.observe]:

```
constexpr T* get_ptr() noexcept;
constexpr const T* get_ptr() const noexcept;
```

Effects: Equivalent to:

```
return has_value() ? std::addressof(val) : nullptr;
```

§ 6.3 Add to [optional.optional.ref.general]:

```
// observers
constexpr T* operator->() const noexcept;
constexpr T& operator*() const noexcept;
constexpr explicit operator bool() const noexcept;
constexpr bool has_value() const noexcept;
constexpr T& value() const; // freestanding-deleted
template<class U = remove_cv_t<T>>
constexpr remove_cv_t<T> value_or(U&& u) const;
constexpr T* get_ptr() const noexcept;
```

§ 6.4 Add to [optional.ref.observe]:

```
constexpr T* get_ptr() const noexcept;
```

Returns: `val` .

§ 6.5 Modify in [version.syn]:

```
#define __cpp_lib_optional 202506LYYYYMMML // also in <optional>
```

§ 7 Acknowledgements

Thanks to the discussion on national body comments for C++26: PL-006, US 150-228, GB 08-225, US 68-122 and US 149-226 for showing this obvious deficiency in `optional` .

§ 8 References

- [N5032](#) Working Draft, Programming Languages - C++
- [N3672](#) A proposal to add a utility class to represent optional objects (Revision 4)
- [P2988](#) `std::optional<T>`

- [P3739](#) Standard Library Hardening - using `std::optional`
- [P3830](#) NB-Commenting is Not a Vehicle for Redesigning `inplace_vector`
- [P3981](#) Better return types in `std::inplace_vector` and `std::exception_ptr_cast`
- [Boost.Optional](#)